Facial Feature Classifica



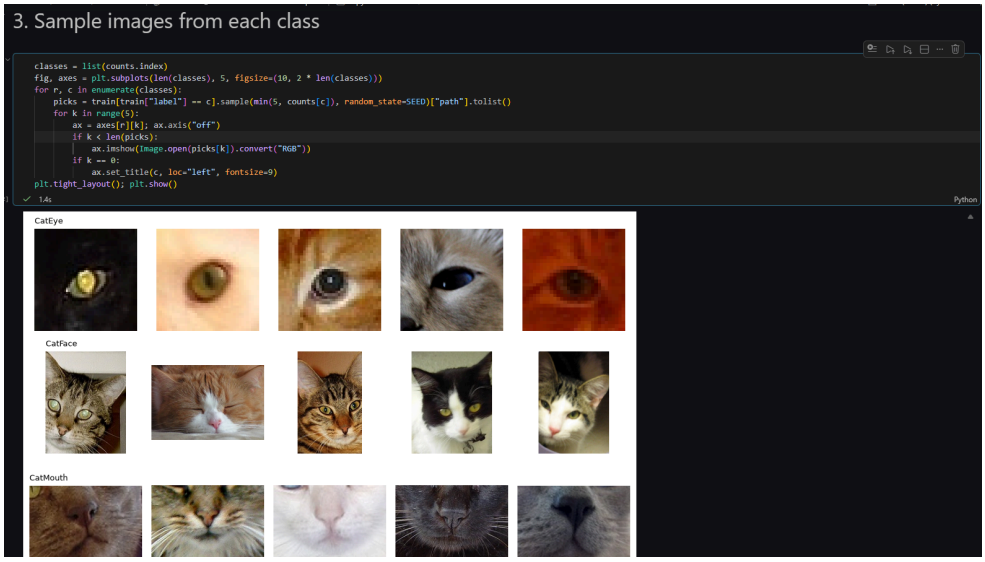
tion Challenge

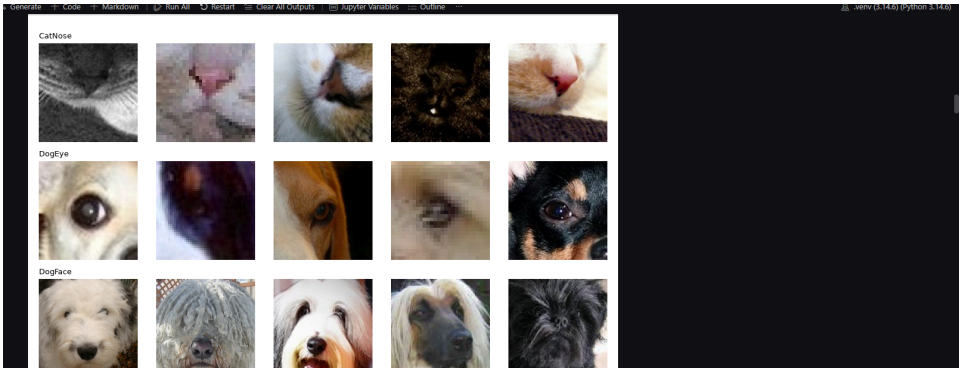
Student name: Matty Luriz ID: 23206450 Username: Outer Skye

Task 1: Dataset Exploration

Dataset and classes

The images in the dataset are stored as JPEG files. There are 63,660 labeled training images and 29,443 unlabeled test images. The images show one of the eight facial features of a cat or a dog. The eight classes are CatEye, CatFace, CatMouth, CatNose, DogEye, DogFace, DogMouth and DogNose. Every training image has a corresponding label and every label has an image file, all of which are valid and can be read.



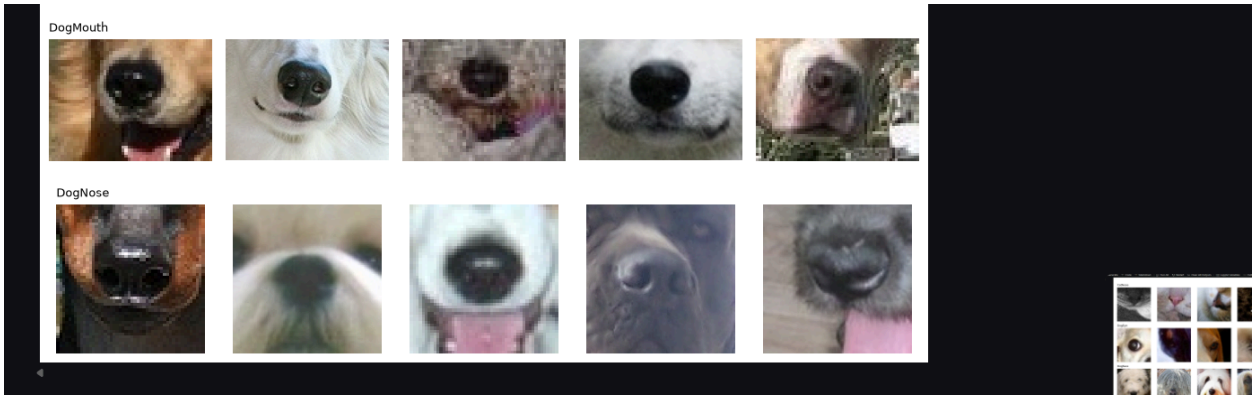


Facial Feature Classifica



tion Challenge

Student name: Matty Luriz ID: 23206450 Username: Outer Skye



Class balance

The classes are extremely imbalanced . The CatEye class has 13,884 images (21.81%) whereas the three classes that represent dogs (DogFace, DogMouth and DogNose) have only 5,790 images each (9.10%). Thus, the largest class accounts for more than twice as many images as the smallest class. Thus, a simple model that always predicts CatEye would already achieve majority-class accuracy of 21.8%. Because of the imbalanced data, stratified split was used in Section 2, and also report per-class accuracy in addition to the overall accuracy for all methods.

Class	Images	Share (%)
CatEye	13,884	21.81
CatFace	6,942	10.90
CatMouth	6,942	10.90
CatNose	6,942	10.90
DogEye	11,580	18.19
DogFace	5,790	9.10

Facial Feature Classifica



tion Challenge

Student name: Matty Luriz ID: 23206450 Username: Outer Skye

DogMouth	5,790	9.10
DogNose	5,790	9.10



Image size

The size of images within the training data ranged from 4x4 pixels up to 2219 x 2219 pixels with 7,861 sizes used in total. The size of the shortest side of the images was found to have a median of 78px. 37.2% of all images were of size less than 64x64 pixels and 69% of all images were square in size. The size of the images was found to vary from class to class with the images of CatFace having a median size of 182 x 215 pixels. The images of DogEye and DogNose had a size of 62 x 62 pixels on average. A simple model that used the width and/or height of the images as input was able to achieve a classification accuracy of 45.8% with 5-fold cross-validation. This is far greater than the majority-class accuracy of 21.8%. The size of the images within the test set followed a similar distribution (median of the shortest side of 77px, 40.4% of less than 64x64 pixels) to that within the training set.

Facial Feature Classifica



tion Challenge

Student name: Matty Luriz ID: 23206450 Username: Outer Skye

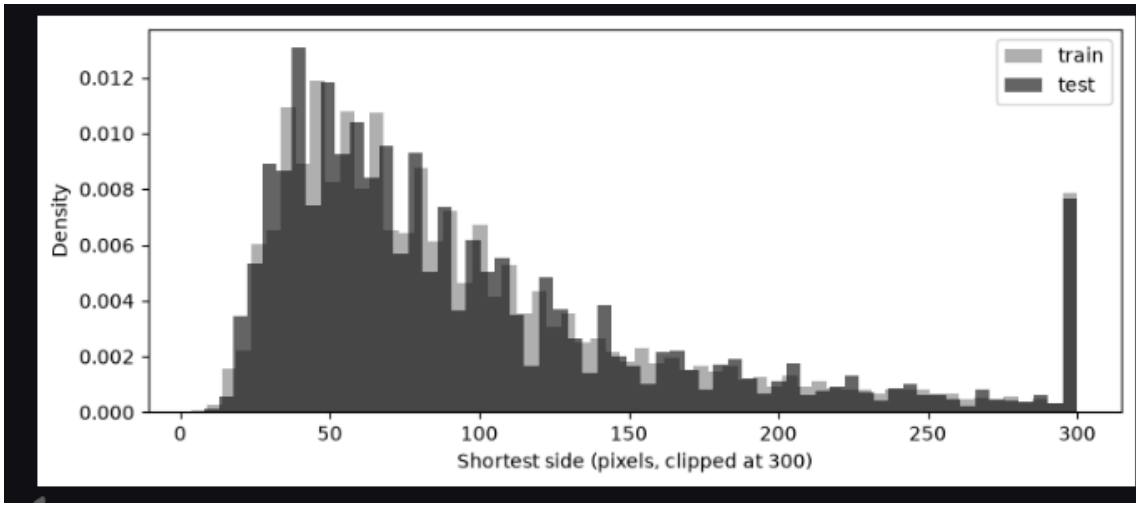
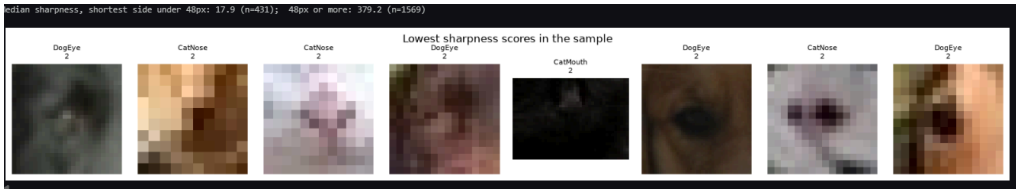


Image quality and duplicates

The small images were often blurry. The sharpness was measured as the variance of a Laplacian filter on a 128 x 128 grayscale copy of the image. Such a sharpness value was determined for a random sample of 2,000 training images. The median value of the sharpness was 17.9 for the images with the shortest side of less than 48 px (n=431) and 379.2 for the images with a shortest side greater than 48 px (n=1,569). Low values denote blurry images.



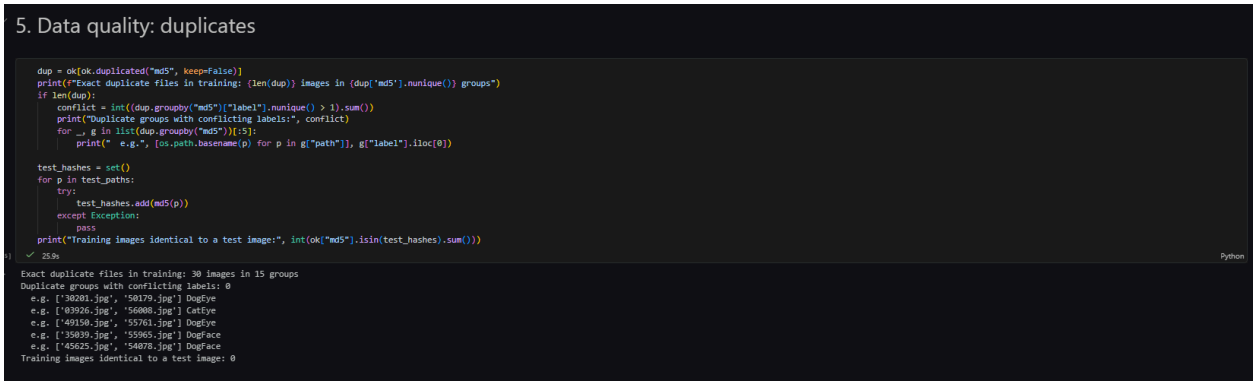
This dataset contains 30 training images. 15 of these images are in exact duplicate form and so form 15 groups of duplicates with identical MD5 hashes. None of these groups contain conflicting labels. No training images are duplicates of test images. There are only a small number of duplicates in total so it is not sensible to put a copy of a duplicate image in the validation set and its twin in the training set as this would lead to overinflation of the accuracy on the validation set. So all of the duplicates are kept together when the data is split.

Facial Feature Classifica



tion Challenge

Student name: Matty Luriz ID: 23206450 Username: Outer Skye



Task 2: Data Preprocessing and Pipeline Preparation

Labels and train/validation split

All labels are assigned integer values from 0 to 7 in alphabetical order of the class names, so that predicted class names can be written out from the integer values. After that the labeled images are split into 80% training set and 20% validation set in stratified random manner, i.e. in such a way that all identical images and images with the same path are in the same set (the function StratifiedGroupKFold from scikit-learn, grouped by file hash, with a fixed random seed). The stratified split ensures that the same class proportions are in the training as well as in the

Resizing and normalisation

Class labels are encoded as integers from 0 to 7 in alphabetical order, so it is easy to write out a classification prediction as a class name. After encoding the labels of the images they were split into 80% training set and 20% validation set. Stratified split was used with duplicates in the original images grouped together. Split is done using scikit-learn's StratifiedGroupKFold class with a random state fixed to a constant value across calls, i.e. hashing of files, so the same

Facial Feature Classifica



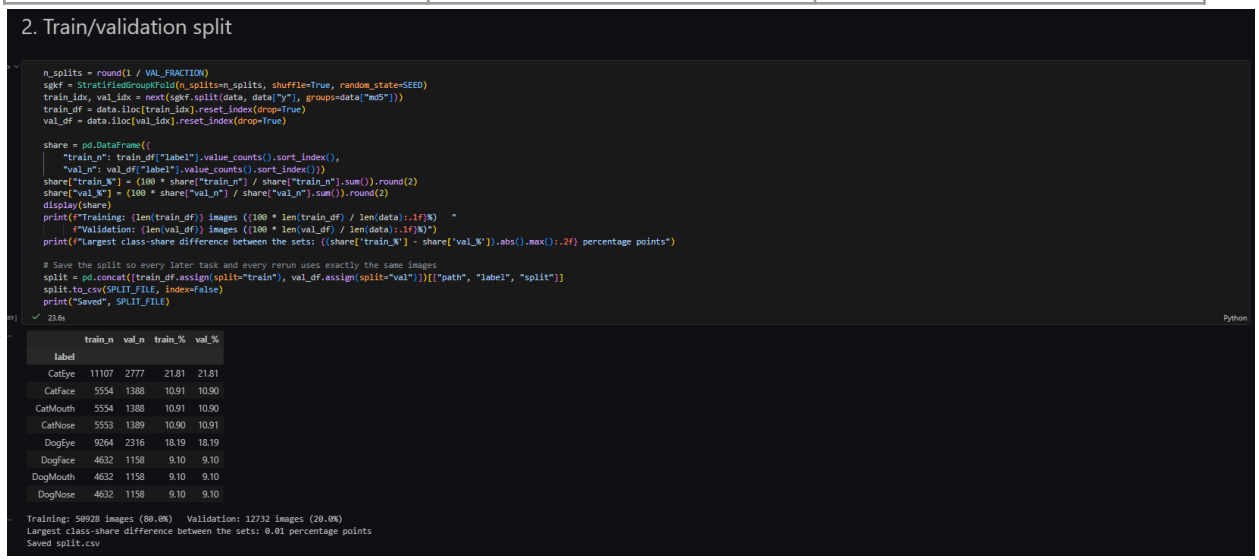
tion Challenge

Student name: Matty Luriz ID: 23206450 Username: Outer Skye

images are used in subsequent runs. The split has 50,928 images in the training set and 12,732 images in the validation set. The largest difference in class share between the training set and the validation set is 0.01 percentage points. The split is saved to a file. It was also verified that there are no identical files and no duplicate image paths in the training and validation sets.

validation set. The grouping by hash also ensures that no duplicate images are in both sets. In total there are 50,928 images in the training set and 12,732 images in the validation set (see Table 2). The biggest difference between the distribution of the classes in the two sets is 0.01%. The split is saved to a file, so that subsequent runs will use the same images (Figure 6). As a final check it is confirmed that there are no identical files and no image path is in both sets.

Data set	Images	Share of labelled data (%)
Training	50,928	80.0
Validation	12,732	20.0
Test (unlabelled)	29,443	not applicable



Facial Feature Classifica

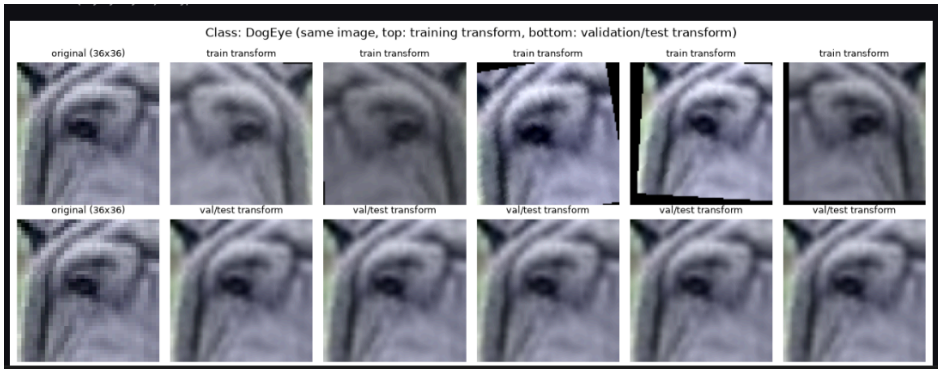


tion Challenge

Student name: Matty Luriz ID: 23206450 Username: Outer Skye

Augmentation

To augment the training set a number of mild transforms have been applied. These include a horizontal flip with a 0.5 probability, random rotation up to 10 degrees, random shift up to 5%, and random scale between 0.9 and 1.1. Additionally there are random changes to the brightness, contrast, and color saturation of the images. This is safe as the shape of the faces (mouth, nose, etc.) are roughly left-right symmetrical. Vertical flips and large rotations have not been used as the images appear to be upright. The validation and test sets have had one transform applied to them in the same way as for the training set but this simply resizes and then normalises the images. An example of the same image going through a training and a validation/test transform.



Data pipeline

The dataset class simply reads the images from disk, converts them to RGB and applies the appropriate transform. The data loaders are set up to use a batch size of 64, shuffle the training set using a fixed seed generator, and use zero workers (because, in Windows notebooks, multiprocessing is extremely unreliable). The test images are loaded in exactly the same order as the sample submission file, and check that every id in the file has a corresponding test image. In the finished pipeline, batches are of shape (64, 3, 96, 96) with values having approximately zero mean and unit standard deviation after normalization.

Facial Feature Classifica



tion Challenge

Student name: Matty Luriz ID: 23206450 Username: Outer Skye

```
train batch: (64, 3, 96, 96)  dtype torch.float32  mean -0.19  std 1.23  min -2.12  max +2.64
val   batch: (64, 3, 96, 96)  dtype torch.float32  mean -0.10  std 1.11  min -2.12  max +2.64
test  batch: (64, 3, 96, 96)  dtype torch.float32  mean -0.06  std 1.10  min -2.12  max +2.64
```

Task 3: Model Selection and Pipeline Development

Model choice

For this experiment ResNet-18 was chosen, pre-trained on ImageNet (He et al., 2016) for transfer learning. ResNet-18 is a residual CNN that has approximately 11.2M parameters and can be trained relatively fast on a laptop CPU (Intel Core i7-10750H, CPU only). Importantly, the edges, textures and shapes that the network has learned from images in ImageNet are expected to transfer to images of cat and dog faces. Importantly, other models such as the Vision Transformer ViT-B/16 (Dosovitskiy et al., 2021) that wasn't used in this experiment has approximately 86M parameters and is much slower when trained on a CPU and expects input of size 224×224. A custom CNN would also need to be trained from scratch and would in all likelihood require more epochs and more data to converge than the ResNet that were pre-trained.

Model Configuration

The input size of the network is 3 x 96 x 96 from the preprocessing stage. The last fully connected layer is replaced with a linear layer of 8 output units. All the other layers use ReLU as the activation function, and no activation function is applied on the output layer in order to get raw scores (logits) of the network output that will be later softened by the softmax function within the loss function. Cross-entropy loss is suitable for single-label classification with 8 output classes. AdamW (Loshchilov & Hutter, 2019) is used as the optimizer with a low learning rate of 3e-4 and a small weight decay of 1e-4 in order to fine-tune the pre-trained weights slowly without 'breaking' them. A fixed batch size of 32 is used in order to get stable estimates of the gradients, and the number of epochs is set to 5 in order to fit the laptop CPU.

Facial Feature Classifica



tion Challenge

Student name: Matty Luriz ID: 23206450 Username: Outer Skye
In the baseline the last residual block (layer4) and the newly added output layer are trained (8.40M out of 11.18M parameters total). Early layers in the network are fixed and detect basic edges and textures that can be transferred to the task at hand, and thus only a portion of the backpropagated gradients need to be computed in each of the 5 epochs. That ran in about 13 minutes per epoch on the laptop's CPU.

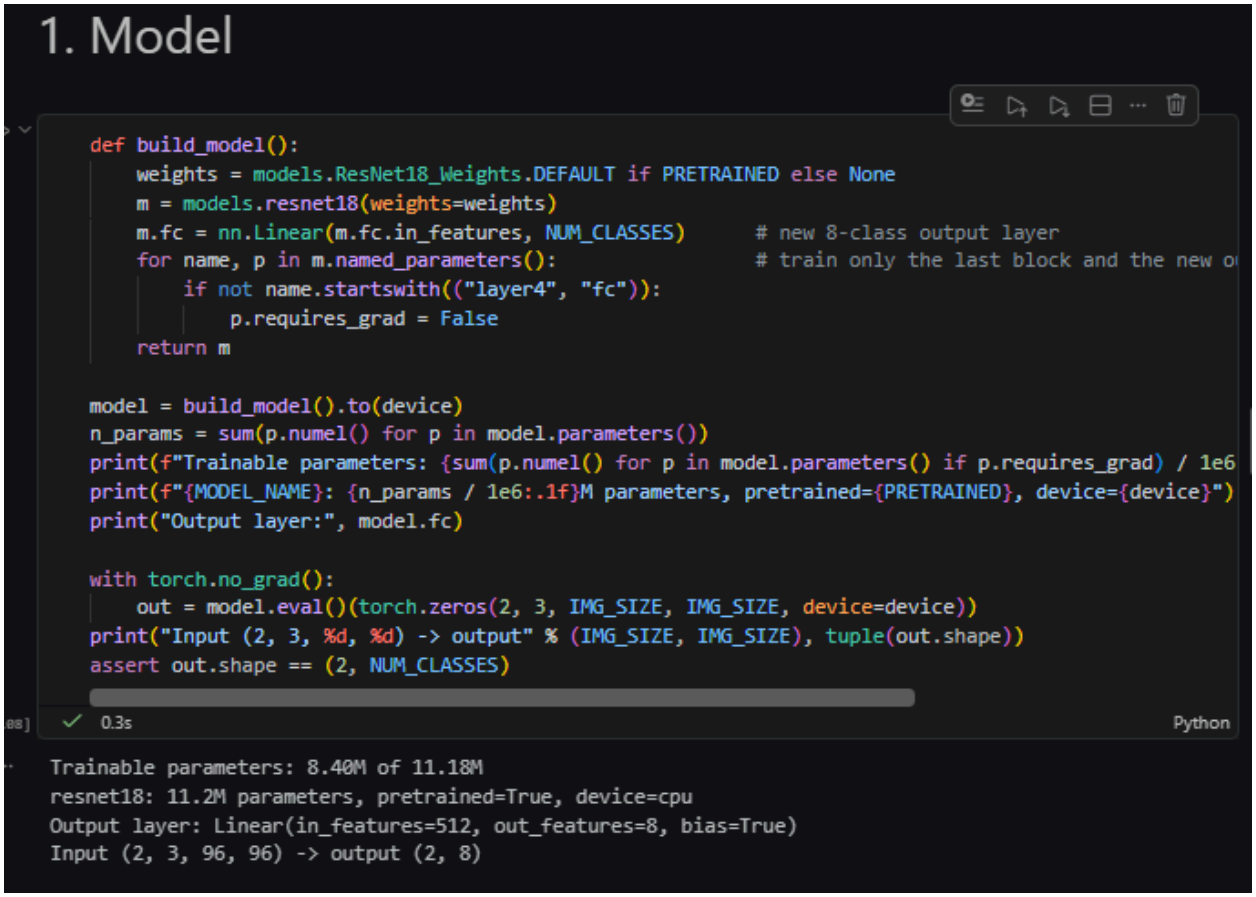
Setting	**Value**
Model	ResNet-18 with ImageNet pretrained weights
Input	3 x 96 x 96, ImageNet normalisation
Output layer	Linear layer, 512 inputs to 8 outputs (logits)
Trainable parts	layer4 and the output layer (8.40M of 11.18M parameters)
Loss	Cross-entropy
Optimiser	AdamW, learning rate 3e-4, weight decay 1e-4
Batch size	32
Epochs	5
Model selection	Weights from the epoch with the best validation accuracy
Hardware	Intel Core i7-10750H, CPU only

Facial Feature Classifica



tion Challenge

Student name: Matty Luriz ID: 23206450 Username: Outer Skye



Training and prediction pipeline

The pipeline consists of 3 functions used to build it. The first function runs one pass over the data in a given data loader (either training or evaluating), returning loss and accuracy. The second function trains the model for a number of epochs, saving out the training and validation loss and accuracy after each epoch. The weights for the model with the best validation accuracy are also saved out. This function can also take in a learning-rate scheduler and early-stopping patience parameters, which are used in Section 4. The third function loads in the weights for the

Facial Feature Classifica



tion Challenge

Student name: Matty Luriz ID: 23206450 Username: Outer Skye
best validation accuracy and uses the model to predict a class for each test image one by one as they are listed in the sample submission file, and the resulting file is checked against sample_submission.csv (column names, number of rows, image IDs in order, class names) before being saved as a file that can be submitted to Kaggle. Before running the full script, we do a speed check by running a few batches and timing how long it takes to complete an epoch, and also include a debug option that runs the whole script on a very small subset of the data first.

Facial Feature Classifica

3. Leakage checks

```
dup_groups = data[data.duplicated("md5", keep=False)]["md5"].nunique()
shared_hashes = set(train_df["md5"]) & set(val_df["md5"])
shared_paths = set(train_df["path"]) & set(val_df["path"])
print(f"Duplicate groups in the labelled data: {dup_groups}")
print("Identical files in both training and validation:", len(shared_hashes))
print("Image paths in both training and validation:", len(shared_paths))
assert not shared_hashes and not shared_paths, "leakage between training and validation"

val_dup_in_train = val_df["md5"].isin(set(train_df["md5"])).sum()
print("Validation images with an exact copy in training:", int(val_dup_in_train))
print("Normalisation constants are fixed ImageNet values, so no statistic is learned from validation or test images.")
```

✓ 0.1s Python

```
Duplicate groups in the labelled data: 15
Identical files in both training and validation: 0
Image paths in both training and validation: 0
Validation images with an exact copy in training: 0
Normalisation constants are fixed ImageNet values, so no statistic is learned from validation or test images.
```

tion Challenge

Student name: Matty Luriz ID: 23206450 Username: Outer Skye

2. Data loaders for training

```
def make_loader(ds, shuffle, seed=SEED):
    if DEBUG_N:
        ds = Subset(ds, range(min(DEBUG_N, len(ds))))
    gen = torch.Generator().manual_seed(seed) if shuffle else None
    return DataLoader(ds, batch_size=TRAIN_BATCH_SIZE, shuffle=shuffle, generator=gen,
                      num_workers=NUM_WORKERS, pin_memory=(device == "cuda"))

fit_train_loader = make_loader(train_ds, shuffle=True)
fit_val_loader = make_loader(val_ds, shuffle=False)
fit_test_loader = make_loader(test_ds, shuffle=False)
print(f"batches per epoch: train {len(fit_train_loader)}, val {len(fit_val_loader)}, test {len(fit
      f"  (batch size {TRAIN_BATCH_SIZE})" + ("  [DEBUG subset]" if DEBUG_N else ""))
```

✓ 0.0s Python

```
batches per epoch: train 1592, val 398, test 921  (batch size 32)
```

4. Speed check

```
# Times a few training batches on a throwaway copy of the model to estimate the length of one epoch
probe = copy.deepcopy(model1).train()
probe_opt = torch.optim.AdamW(probe.parameters(), lr=LEARNING_RATE)
probe_scaler = torch.amp.GradScaler(device, enabled=use_amp)
n_probe, t0 = 0, None
for i, (xb, yb) in enumerate(fit_train_loader):
    if i == 2:                          # skip the first batches (warm-up)
        if device == "cuda":
            torch.cuda.synchronize()
        t0 = time.time()
    xb, yb = xb.to(device), yb.to(device)
    with torch.autocast(device_type=device, enabled=use_amp):
        loss = nn.functional.cross_entropy(probe(xb), yb)
    probe_opt.zero_grad(set_to_none=True)
    probe_scaler.scale(loss).backward(); probe_scaler.step(probe_opt); probe_scaler.update()
    if i >= 2:
        n_probe += 1
    if i == 11 or i == len(fit_train_loader) - 1:
        break
if device == "cuda":
    torch.cuda.synchronize()
if n_probe:
    per_batch = (time.time() - t0) / n_probe
    print(f"{per_batch:.2f}s per batch -> about {per_batch * len(fit_train_loader) / 60:.1f} min per training epoch "
          f"(plus validation), about {per_batch * len(fit_train_loader) * EPOCHS / 60:.0f} min for {EPOCHS} epochs")
if device == "cuda":
    print(f"peak GPU memory: {torch.cuda.max_memory_allocated() / 2**30:.2f} GiB of {torch.cuda.get_device_properties(0).total_memory / 2**30:.1f} GiB")
del probe, probe_opt
```

✓ 5.2s Python

```
0.42s per batch -> about 11.2 min per training epoch (plus validation), about 56 min for 5 epochs
```

Facial Feature Classifica



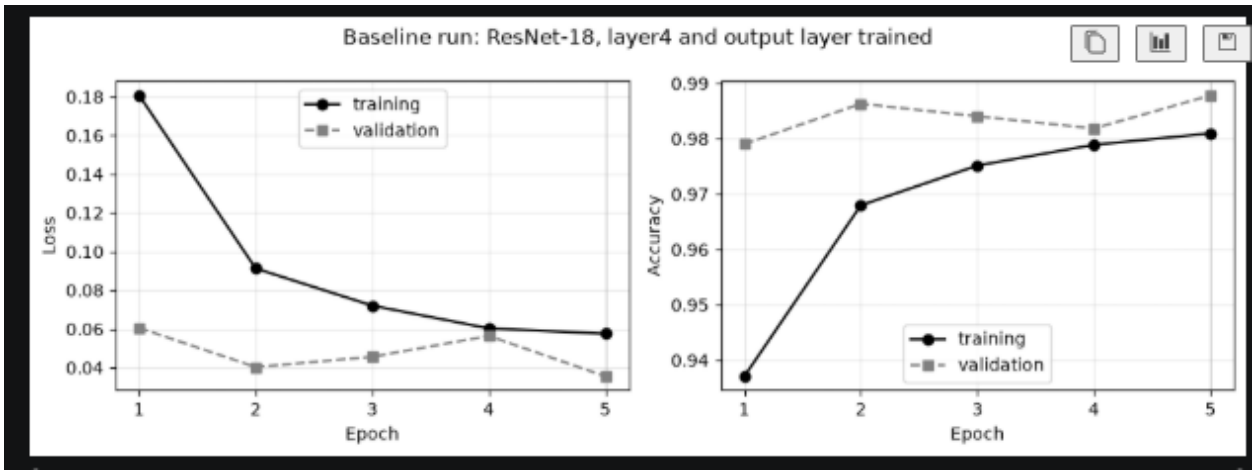
tion Challenge

Student name: Matty Luriz ID: 23206450 Username: Outer Skye

Task 4: Training Strategy and Overfitting Management

Baseline learning curves

For the baseline run in Section 3, 5 epochs were used to train the network for and plot the learning curves. The training accuracy is increasing from 93.7% to 98.1%, and the training loss is decreasing in every epoch from 0.181 to 0.058. The validation accuracy is increasing from 97.9% to 98.78% in epoch 5 where the validation loss is also at its lowest (0.0355).



This run of the network does not appear to overfit the training data. The characteristic sign of overfitting is that the validation loss will increase while the training loss continues to decrease. Moreover, the training accuracy will be greater than the validation accuracy. As was previously stated, the accuracy and loss values for training are averaged over the entire epoch while the weights are still changing. The images used for these averages have been randomly augmented. The accuracy and loss values for validation on the other hand are recorded using the images in their clean form and the values recorded are for the weights at the end of the epoch. In this run, the validation loss increased slightly in epochs 3 and 4 but decreased in epoch 5.

Facial Feature Classifica



tion Challenge

Student name: Matty Luriz ID: 23206450 Username: Outer Skye

There are two things going on here. First, it looks like mild underfitting. Since the loss is still going down on the training data and the best epoch on the validation data was the last one, there is probably more to learn from this dataset. Second, the fact that only layer4 and the output layer are trainable limits the ability of the network to fit this dataset. The fluctuations in the validation loss may indicate that the constant learning rate is too high for the later epochs.

Strategies applied

For these curve changes, the concern is with underfitting and noisy validation rather than with overfitting, and made these changes as a group of three improvements that ran as one improvement.
The first improvement changes the learning rate schedule to a Cosine annealing schedule (Loshchilov & Hutter, 2017) decreasing from 3e-4 to 0 over the 5 epochs of training. Increasing the step size of the learning rate in the early stages of training helps to increase the pace of progress through the stages of training, and then decreasing the step size in the later stages helps to decrease the fluctuation in validation loss. The second improvement switches to fine-tuning the later layers of the network: the training loss is still decreasing and there is no sign of overfitting, so it would seem to make sense to increase the number of trainable parameters in the network. The later layers of a pre-trained network are most specific to the pre-training task, so the earliest layers are less likely to help with fine-tuning than the later layers. So, layer3 and layer4 are fine-tuned, increasing the number of trainable parameters from 8.40M to 10.50M out of 11.18M.

The following hyperparameters have not been modified: weight decay (1e-4) for AdamW, training-only augmentation, batch normalisation in the ResNet-18 model, and the model has been started from the pre-trained weights for the baseline model. All of the following have not been added to the model as they would have introduced underfitting and would have slowed down learning: dropout, stronger data augmentation. The improvement in performance of the two changes (i.e. learning-rate scheduling and extra trainable parameters in the later layers) cannot be established from a single run (which takes over an hour on a laptop CPU).

Facial Feature Classifica



tion Challenge

Student name: Matty Luriz ID: 23206450 Username: Outer Skye

Setting	Baseline	Improvement run
Trainable parts	layer4 and output layer (8.40M)	layer3, layer4 and output layer (10.50M)
Learning rate	3e-4, constant	3e-4, cosine annealing (2.9e-5 in epoch 5)
Early stopping	None	Patience of 2 epochs (not triggered)
Epochs, batch size, weight decay	5, 32, 1e-4	5, 32, 1e-4 (unchanged)

Results of the improvement run

The best validation accuracy achieved by the improvement run was 99.47% in epoch 5 with a validation loss of 0.0152. Also the validation accuracy and loss improved in all epochs (from 98.94% and 0.0325 in epoch 1 to 99.47% and 0.0152 in epoch 5). In contrast to the baseline the validation loss did not increase in any epoch. The training accuracy reached 99.51% at the end which is close to the best validation accuracy. So the difference between validation and training accuracy is about zero which does not indicate overfitting so far. Early stopping was not

Facial Feature Classifica

3. Leakage checks

```
dup_groups = data[data.duplicated("md5", keep=False)]["md5"].nunique()
shared_hashes = set(train_df["md5"]) & set(val_df["md5"])
shared_paths = set(train_df["path"]) & set(val_df["path"])
print(f"Duplicate groups in the labelled data: {dup_groups}")
print("Identical files in both training and validation:", len(shared_hashes))
print("Image paths in both training and validation:", len(shared_paths))
assert not shared_hashes and not shared_paths, "leakage between training and validation"

val_dup_in_train = val_df["md5"].isin(set(train_df["md5"])).sum()
print("Validation images with an exact copy in training:", int(val_dup_in_train))
print("Normalisation constants are fixed ImageNet values, so no statistic is learned from validation or test images.")
```

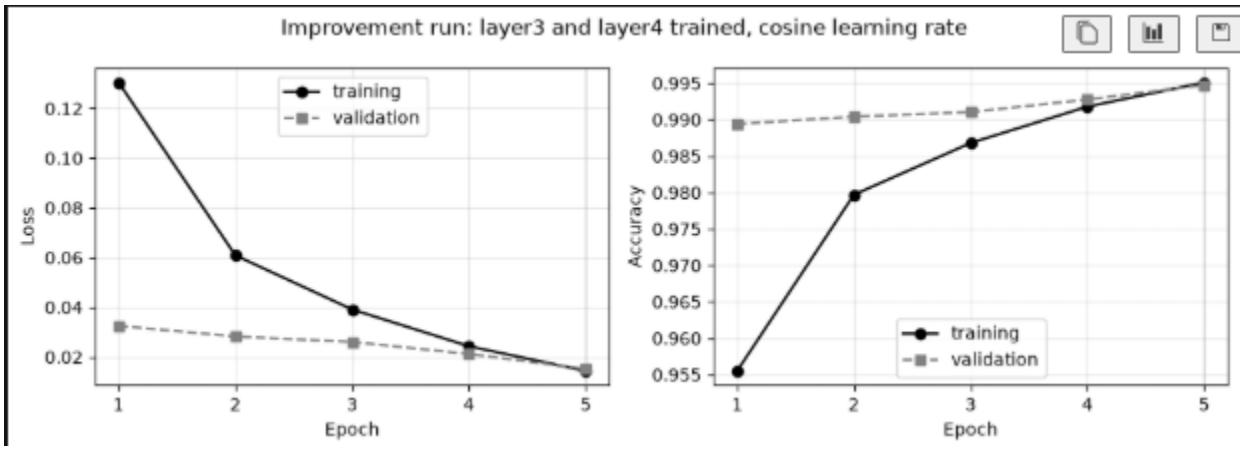
✓ 0.1s Python

```
Duplicate groups in the labelled data: 15
Identical files in both training and validation: 0
Image paths in both training and validation: 0
Validation images with an exact copy in training: 0
Normalisation constants are fixed ImageNet values, so no statistic is learned from validation or test images.
```

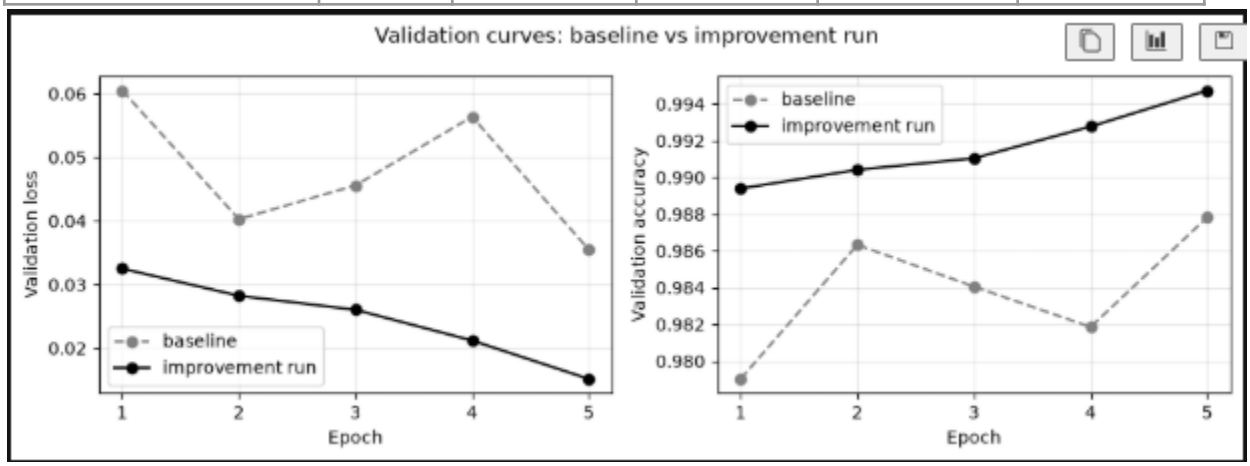
tion Challenge

Student name: Matty Luriz ID: 23206450 Username: Outer Skye

activated since the validation accuracy increased in all epochs.



Run	Best epoch	Validation accuracy	Validation loss	Training accuracy	Minutes per epoch
Baseline	5	0.9878	0.0355	0.9810	12.8
Improvement run	5	0.9947	0.0152	0.9951	14.0



Facial Feature Classifica

3. Leakage checks

```
dup_groups = data[data.duplicated("md5", keep=False)]["md5"].nunique()
shared_hashes = set(train_df["md5"]) & set(val_df["md5"])
shared_paths = set(train_df["path"]) & set(val_df["path"])
print(f"Duplicate groups in the labelled data: {dup_groups}")
print("Identical files in both training and validation:", len(shared_hashes))
print("Image paths in both training and validation:", len(shared_paths))
assert not shared_hashes and not shared_paths, "leakage between training and validation"

val_dup_in_train = val_df["md5"].isin(set(train_df["md5"])).sum()
print("Validation images with an exact copy in training:", int(val_dup_in_train))
print("Normalisation constants are fixed ImageNet values, so no statistic is learned from validation or test images.")
```

✓ 0.1s Python

```
Duplicate groups in the labelled data: 15
Identical files in both training and validation: 0
Image paths in both training and validation: 0
Validation images with an exact copy in training: 0
Normalisation constants are fixed ImageNet values, so no statistic is learned from validation or test images.
```

tion Challenge

Student name: Matty Luriz ID: 23206450 Username: Outer Skye
The results show that all changes were in the right direction. Unfortunately, the separate effects were not tested. The extra layer is expensive in terms of trainable parameters and hence time per epoch: 12.8 minutes vs 14.0 minutes (ca. 9% increase for a ca. 25% increase in the number of parameters) for the time on a laptop CPU. However, the validation loss was still going down at epoch 5, so it would be interesting to see whether one could get even better results by running for more epochs.

Task 5: Model Evaluation and Kaggle submission

Validation process

The best model from the improvement runs is the one that was trained with the best weights for the corresponding validation run (of 12,732 images). For the 12,732 validation images this model reached an accuracy of 99.47%, a loss of 0.0152 and a macro F1-score of 0.9948. The accuracy is the main metric that Kaggle uses to rank submissions. The model also was evaluated for precision, recall and F1-score of all classes. The implementation of these metrics for an imbalanced set of classes was based on scikit-learn (Pedregosa et al., 2011). The recall value for a class equals its accuracy value.

1. Validation predictions

```
final_model = load_trained(FINAL_WEIGHTS)

@torch.no_grad()
def predict_validation(model, loader):
    losses, y_true, y_pred = 0.0, [], []
    n = 0
    for xb, yb in loader:
        xb, yb = xb.to(device), yb.to(device)
        with torch.autocast(device_type=device, enabled=use_amp):
            logits = model(xb)
        losses += nn.functional.cross_entropy(logits.float(), yb, reduction="sum").item()
        y_true += yb.cpu().tolist()
        y_pred += logits.argmax(1).cpu().tolist()
        n += len(yb)
    return losses / n, np.array(y_true), np.array(y_pred)

val_loss, y_true, y_pred = predict_validation(final_model, fit_val_loader)
val_acc = float((y_true == y_pred).mean())
prec, rec, f1, support = precision_recall_fscore_support(y_true, y_pred, labels=list(range(NUM_CLASSES)), zero_division=0)
print(f"Validation images: {len(y_true)}")
print(f"Validation accuracy: {val_acc:.4f}   validation loss: {val_loss:.4f}   macro F1: {f1.mean():.4f}")

history_final = pd.read_csv(HISTORY_FILE_T4)
print(f"Best epoch in {HISTORY_FILE_T4}: validation accuracy {history_final['val_acc'].max():.4f}, "
      f"validation loss {history_final.loc[history_final['val_acc'].idxmax(), 'val_loss']:.4f}  (should match the line above)")
```

✓ 1m 50.5s Python

```
Validation images: 12732
Validation accuracy: 0.9947   validation loss: 0.0152   macro F1: 0.9948
Best epoch in history_task4.csv: validation accuracy 0.9947, validation loss 0.0152  (should match the line above)
```

Facial Feature Classifica



tion Challenge

Student name: Matty Luriz ID: 23206450 Username: Outer Skye

Class	Precision	Recall	F1-score	Images
CatEye	0.9953	0.9953	0.9953	2,777
CatFace	1.0000	0.9993	0.9996	1,388
CatMouth	0.9843	0.9906	0.9874	1,388
CatNose	0.9891	0.9820	0.9855	1,389
DogEye	0.9948	0.9965	0.9957	2,316
DogFace	0.9991	1.0000	0.9996	1,158
DogMouth	0.9974	0.9991	0.9983	1,158
DogNose	0.9991	0.9948	0.9970	1,158

This code was submitted to the competition on Kaggle under the user name Outer Skye. It reaches a score of 0.9723 on the public test data which is 0.61 percentage points better than the score of the baseline submission (0.9662).

Confusion Matrix

Out of 12,732 validation images 67 were misclassified (Figure 15). The most common errors were CatNose classified as CatMouth (20 images) and CatMouth classified as CatNose (10 images), together accounting for 30 errors out of 67. Errors between CatEye and DogEye (8 images each way) misclassified species but correct facial feature, while the rest of errors occurred lat most 4 times for each of the different pairs.

Facial Feature Classifica

3. Leakage checks

```
dup_groups = data[data.duplicated("md5", keep=False)]["md5"].nunique()
shared_hashes = set(train_df["md5"]) & set(val_df["md5"])
shared_paths = set(train_df["path"]) & set(val_df["path"])
print(f"Duplicate groups in the labelled data: {dup_groups}")
print("Identical files in both training and validation:", len(shared_hashes))
print("Image paths in both training and validation:", len(shared_paths))
assert not shared_hashes and not shared_paths, "leakage between training and validation"

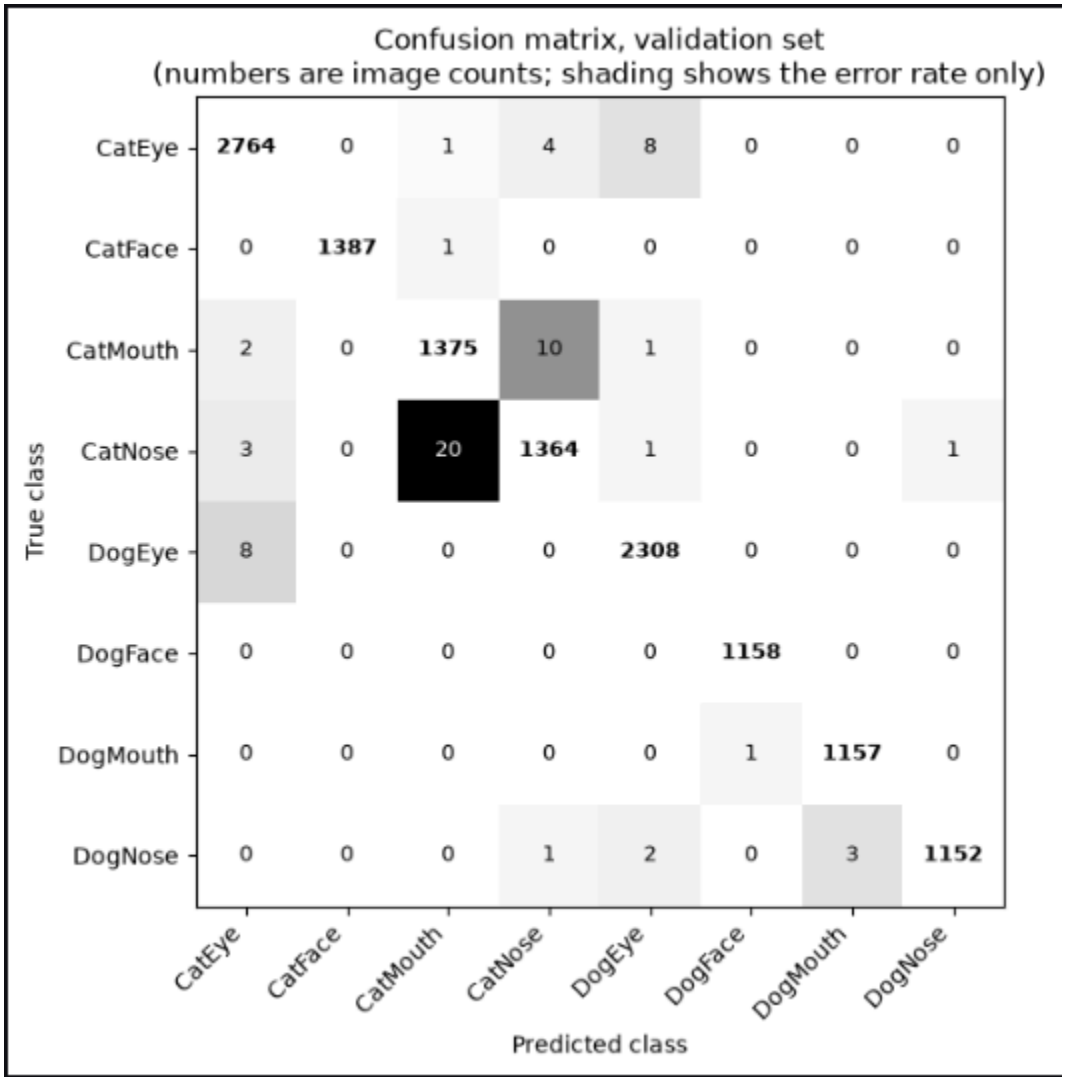
val_dup_in_train = val_df["md5"].isin(set(train_df["md5"])).sum()
print("Validation images with an exact copy in training:", int(val_dup_in_train))
print("Normalisation constants are fixed ImageNet values, so no statistic is learned from validation or test images.")
```

✓ 0.1s Python

```
Duplicate groups in the labelled data: 15
Identical files in both training and validation: 0
Image paths in both training and validation: 0
Validation images with an exact copy in training: 0
Normalisation constants are fixed ImageNet values, so no statistic is learned from validation or test images.
```

tion Challenge

Student name: Matty Luriz ID: 23206450 Username: Outer Skye



Facial Feature Classifica

3. Leakage checks

```
dup_groups = data[data.duplicated("md5", keep=False)]["md5"].nunique()
shared_hashes = set(train_df["md5"]) & set(val_df["md5"])
shared_paths = set(train_df["path"]) & set(val_df["path"])
print(f"Duplicate groups in the labelled data: {dup_groups}")
print("Identical files in both training and validation:", len(shared_hashes))
print("Image paths in both training and validation:", len(shared_paths))
assert not shared_hashes and not shared_paths, "leakage between training and validation"

val_dup_in_train = val_df["md5"].isin(set(train_df["md5"])).sum()
print("Validation images with an exact copy in training:", int(val_dup_in_train))
print("Normalisation constants are fixed ImageNet values, so no statistic is learned from validation or test images.")
```

✓ 0.1s Python

```
Duplicate groups in the labelled data: 15
Identical files in both training and validation: 0
Image paths in both training and validation: 0
Validation images with an exact copy in training: 0
Normalisation constants are fixed ImageNet values, so no statistic is learned from validation or test images.
```

tion Challenge

Student name: Matty Luriz ID: 23206450 Username: Outer Skye

Kaggle submission

The predictions for the 29,443 test images were generated with the final model and written to submission_task4.csv, which was then checked against sample_submission.csv for column names, number of rows, image ids in submission order, and valid class names before finally submitting the file (see Figure 16). The test set class shares were close to the corresponding training set shares for all classes with an error of less than 0.6 percentage points. Only for the class CatNose the model predicted this class 1.06 percentage points less frequently than it was expected according to its test set predictions of 9.84% (10.90% for the training set). Note that the class mix for the test set was not known, so this observation is purely of academic interest.

4. Kaggle submission file

```
final_sub = pd.read_csv(FINAL_SUBMISSION)
assert list(final_sub.columns) == list(sub.columns), "columns must match the sample submission"
assert len(final_sub) == len(sub), (len(final_sub), len(sub))
assert final_sub["image_id"].tolist() == sub["image_id"].tolist(), "ids must match the sample submission order"
assert set(final_sub["class"]) <= set(classes), "unknown class name in the submission"
assert final_sub.notna().all().all(), "missing values in the submission"
print(f"{FINAL_SUBMISSION}: {len(final_sub)} rows, columns {list(final_sub.columns)}, format checked against sample_submission.csv")

mix = pd.DataFrame({
    "training %": (100 * train_df["label"].value_counts(normalize=True)).round(2),
    "validation %": (100 * val_df["label"].value_counts(normalize=True)).round(2),
    "predicted test %": (100 * final_sub["class"].value_counts(normalize=True)).round(2)}).loc[classes]
display(mix)
```

✓ 0.0s Python

```
submission_task4.csv: 29443 rows, columns ['image_id', 'class'], format checked against sample_submission.csv
```

	training %	validation %	predicted test %
CatEye	21.81	21.81	21.62
CatFace	10.91	10.90	10.57
CatMouth	10.91	10.90	11.11
CatNose	10.90	10.91	9.84
DogEye	18.19	18.19	18.60
DogFace	9.10	9.10	9.59
DogMouth	9.10	9.10	9.69
DogNose	9.10	9.10	8.98

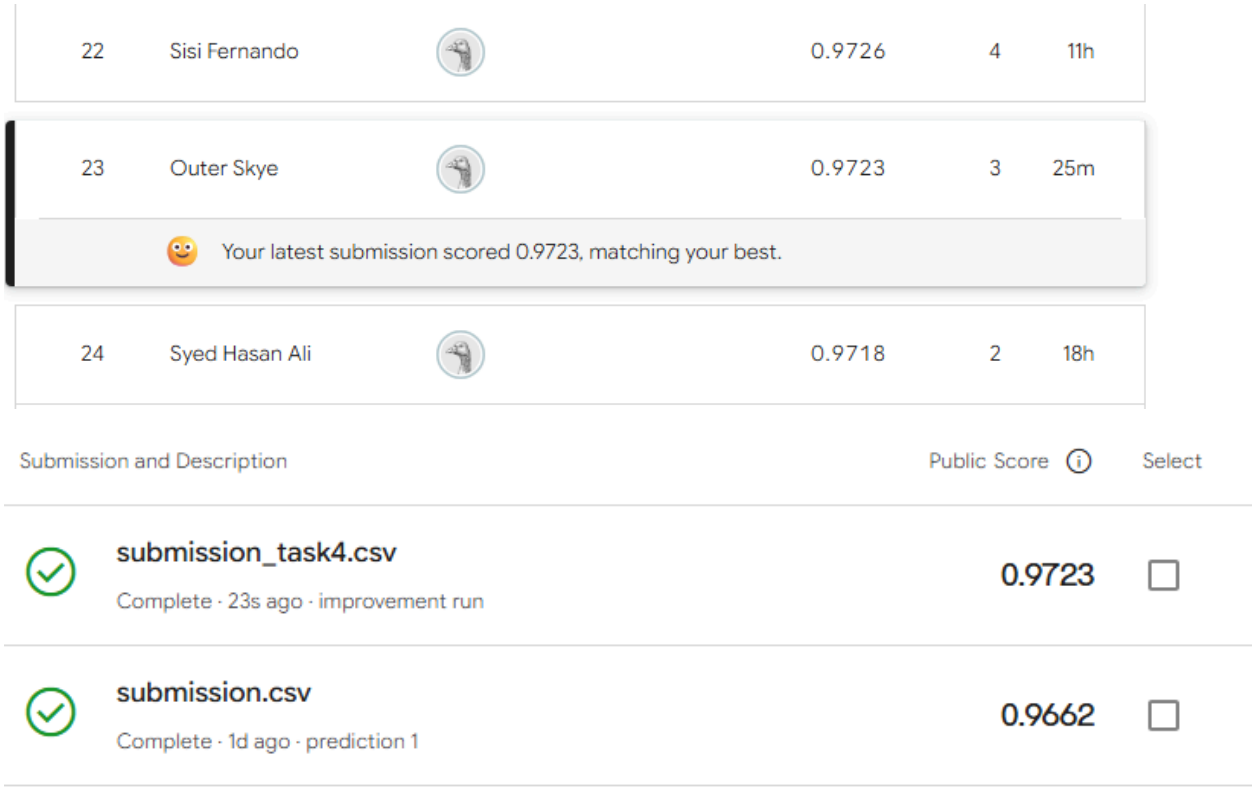
The improved version was uploaded as Outer Skye to Kaggle and scored 0.9723 on the public test data, ranking 23rd out on the public Kaggle leaderboard at the time of these screenshots. The improvement was therefore 0.61 percentage points above the baseline submission of 0.9662.

Facial Feature Classifica



tion Challenge

Student name: Matty Luriz ID: 23206450 Username: Outer Skye



Validation compared with the kaggle leaderboard

The table below presents a comparison of the model's score on the validation set and on the test set (Kaggle) for both the baseline and improved versions of the model. The score on the test set was roughly 2.2% points below the score on the validation set for both versions of the model. The gain in the improved version of the model was largely consistent between validation and test (0.69pt on validation and 0.61pt on test). So while the extra capacity in the improved model as well as the learning-rate schedule did not even decrease the gap between validation and test for the improved model, the gain in itself did carry over.

Facial Feature Classifica



tion Challenge

Student name: Matty Luriz ID: 23206450 Username: Outer Skye

Model	Validation accuracy	Validation loss	Kaggle public score	Difference (points)
Baseline	0.9878	0.0355	0.9662	2.16
Improvement run	0.9947	0.0152	0.9723	2.24

There are a number of possible reasons for this size gap. The validation images are selected from the same set of labeled images as used for training, so near-duplicate images could easily be split between the validation set and the training set. The best epoch for each model was chosen using validation accuracy on the validation images (which are slightly optimistic given the images used for validation and for testing on the public score are slightly different). The public score is noisier than the validation score as it is only calculated on a portion of the test set. Finally, the test set may contain a larger amount of images of very small size rather than the training set in the training set (Section 1 found that 8.2% of test images were of size under 32px compared to 6.5% in the training set of images with labels). These potential explanations for the size gap were not tested here.